

上机作业 1：数值计算中的误差 —— 实验报告

1. 基本信息

- 姓名：郭如诚
- 学号：1120250999
- 学号最后两位 $s = 99$
- 个人参数 $p = (99 \bmod 8) + 1 = 4$

2. 任务 1：计算顺序与浮点误差

运行前预测

取 $a = 10^{16}$, $b = -a$, $c = 1$:

- $r_1 = (a + b) + c = 1$
- $r_2 = a + (b + c) = 0$

预测：数学上两者都等于 1，但预测计算机得到的结果会不相同（ r_2 会得到 0）。

理由：双精度浮点数只有约 16 位有效数字，当 a 很大时， $b + c = -a + 1$ 中的 1 相对 a 太小，会被舍掉，导致 $r_2 = 0$ ；而 $(a + b)$ 先抵消为 0，再加 c 得到 1。

结果表

a	$(a + b) + c$	$a + (b + c)$	是否相同
10^{10}	$1.000000000000 \times 10^0$	$1.000000000000 \times 10^0$	相同
10^{12}	$1.000000000000 \times 10^0$	$1.000000000000 \times 10^0$	相同
10^{14}	$1.000000000000 \times 10^0$	$1.000000000000 \times 10^0$	相同
10^{16}	$1.000000000000 \times 10^0$	0	不同
10^{18}	$1.000000000000 \times 10^0$	0	不同
4×10^{16} (个人值 a_p)	$1.000000000000 \times 10^0$	0	不同

结果分析

(1) 从 $a = 10^{16}$ 开始两种计算顺序表现出差异；我的个人测试值 $a_p = 4 \times 10^{16}$ 也是 $(a+b)+c = 1.0$ 而 $a+(b+c) = 0.0$ ，结果不同。

(2) 双精度浮点数 (float64) 的有效数字约 15~16 位十进制数字，这是一种有限精度表示；

当 $a \geq 10^{16}$ 时，计算 $b + c = -a + 1$ 需要把 1 附加到 $-a$ 的第 16 位有效数字之后，超出了可表示的范围，1 被舍入掉，结果仍是 $-a$ ，即"大数吃小数"；

于是 $a + (b + c) = a + (-a) = 0$ ，产生了舍入误差；

而 $(a + b) + c$ 先做同量级相消得 0，再加 1，没有小数被吃掉，结果正确。这说明浮点加法不满足结合律，计算顺序会影响结果。

3. 任务 2：相近数相减与有效数字损失

运行前预测

选择：B (y_2 更可靠)。

理由： x 增大时， $\sqrt{x+1}$ 与 $\sqrt{x}$ 会十分接近， y_1 要做两个相近大数的减法，高位有效数字相互抵消，存储中的舍入误差被暴露出来，有效数字大量损失； x 大到一定程度（如 10^{16} ）时， $\sqrt{x+1}$ 和 $\sqrt{x}$ 在计算机里甚至无法区分， y_1 会直接算出 0，计算失效。而 y_2 的分母是加法 $\sqrt{x+1} + \sqrt{x}$ ，两个正数相加不会相消、不会损失有效数字，再做一次除法即可，所以 y_2 更可靠。

结果表

x	$y_1 = \sqrt{x+1} - \sqrt{x}$	$y_2 = 1/(\sqrt{x+1} + \sqrt{x})$	$ y_1 - y_2 $
10^2	$4.987562112089 \times 10^{-2}$	$4.987562112089 \times 10^{-2}$	$3.261280134836 \times 10^{-16}$
10^4	$4.999875006249 \times 10^{-3}$	$4.999875006250 \times 10^{-3}$	$1.065120214250 \times 10^{-15}$
10^8	$5.000000055588 \times 10^{-5}$	$4.999999987500 \times 10^{-5}$	$6.808831656043 \times 10^{-13}$
10^{12}	$5.000038072467 \times 10^{-7}$	$4.999999999999 \times 10^{-7}$	$3.807246810094 \times 10^{-12}$
10^{16}	0	$5.000000000000 \times 10^{-9}$	$5.000000000000 \times 10^{-9}$
4×10^{12} (个人值 x_p)	$2.500601112843 \times 10^{-7}$	$2.500000000000 \times 10^{-7}$	$6.011128427161 \times 10^{-11}$

结果分析

(1) 随着 x 增大，两种算法的差异越来越大，差异越来越明显： $x = 10^2$ 时差异只有 10^{-16} 量级； $x = 10^{12}$ 时 y_1 的第 5 位有效数字已经出错；到 $x = 10^{16}$ 时 y_1 直接算出 0（此时 $\sqrt{x+1}$ 与 $\sqrt{x}$ 已无法区分），与 y_2 的差等于真

值本身，即 y_1 完全失效。我的个人测试值 $x_p = 4 \times 10^{12}$ 时， $y_1 = 2.500601 \times 10^{-7}$ 在第 4 位有效数字上就已偏离 $y_2 = 2.500000 \times 10^{-7}$ ，同样印证了“ x 越大 y_1 越不可靠”。

(2) $\sqrt{x+1}$ 与 $\sqrt{x}$ 都是在 $\sqrt{x}$ 量级左右的数，而它们的差约为 $1/(2\sqrt{x})$ ，是个很小的数。对于保存不了的数字，计算机只好在“能表示的数”里挑一个离真值最近的存起来。两个相近数相减时，两者高位相同的有效数字相互抵消，只剩下前面存储的近似值中的舍入误差，导致结果的有效数字大量损失， x 越大相消越严重，直至结果只剩舍入误差甚至为 0。

(3) 数学上等价的公式只保证在无限精度意义下结果相同。计算机中每个数都是有限精度近似，每次运算都要舍入，不同的运算路径上舍入误差的产生和传播方式不一样： y_1 把误差放在“相近数相减”这个上， y_2 的路径只有加法和除法。因此数学上等价公式在计算机上的计算效果可以完全不同。

4. 任务 3：数学等价公式与数值稳定性

个人参数： $K = p \times 10^9 = 4 \times 10^9$ 。

输入数据实际存储值

- $K = 4 \times 10^9$
- $K + 1$ （数学值）= 4000000001
- $\text{float32}(K + 1) = 4 \times 10^9$ （与 K 相同）

float32 只有约 7 位有效数字，4000000001 的第 10 位“1”无法表示，被舍入掉，所以 float32 中 $K + 1$ 与 K 存储为同一个数；相应地 $b = -(K+1)$ 实际存成 -4×10^9 。

方法 A、B 结果表 (numpy.float32)

方法	x_1	x_2	x_2 的相对误差
方法 A	4.0000000000×10^9	0	1.000000×10^0 (100%)
方法 B	4.0000000000×10^9	1.0000000000×10^0	0

结果分析

(1) 不相同。float32 中实际存储的 $K + 1$ 是 4×10^9 ，大小与 K 相同，与数学上的 4000000001 相差 1。是数据表示误差——输入系数在进入计算之前就已经不准了。

(2) 方法 A 的精度损失有两个环节：其一，输入 $b = -(K+1)$ 在 float32 中被舍入为 $-K$ （表示误差）；其二，计算小根 $x_2 = (-b - \sqrt{b^2 - 4ac}) / 2a$ 时， $-b$ 与 $\sqrt{\quad}$ 判别式都是约 4×10^9 的相近大数，两者相减发生抵消，分子直接变成 0，于是 $x_2 = 0$ ，相对误差 100%。本例中运算环节的相近数相减把结果变为 0。

(3) 方法 B 先用"避免相消"的符号选取公式算出大根 x_1 ($-b$ 与 $\sqrt{\cdot}$ 同号相加, 不相消), 再利用韦达定理 $x_2 = c/(a \cdot x_1)$ 求小根——一次乘除法, 完全不涉及相近数相减。它仍用 float32, 只是重新组织了运算顺序、避开了误差放大环节, 所以能得到 $x_2 = 1$ 的精确结果。通过选择稳定算法来改善结果。

(4) "公式正确"不等于"计算方法数值稳定"。方法 A 的求根公式在数学上完全正确, 但在有限精度下它让相近数相减, 舍入误差被急剧放大, 给出 100% 错误的结果; 方法 B 同样基于正确的数学 (求根公式 + 韦达定理), 但运算路径稳定。公式正确只保证无限精度下成立, 数值稳定性考察的是舍入误差在具体运算路径上的传播。

5. 任务 4：良态问题、病态问题与算法稳定性

(1) 个人参数: $\epsilon = p \times 10^{-6} = 4 \times 10^{-6}$ 。

结果表

问题	输入相对变化 r_x	输出相对变化 r_y
$y = 1/x \ (x = 2 \rightarrow 2+\epsilon)$	$2.000000000058 \times 10^{-6}$	$1.999996000035 \times 10^{-6}$
$y = 1/(x-1) \ (x = 1+\epsilon \rightarrow 1+2\epsilon)$	$3.999984000179 \times 10^{-6}$	$5.000000000139 \times 10^{-1}$

结果分析

(2) 问题 B 对输入数据的微小变化更加敏感: 输入相对变化约 4×10^{-6} , 输出相对变化为 0.5, 放大约 12.5 万倍; 而问题 A 的输出变化与输入变化几乎一样大。

(3) 选择: C (数学问题本身对数据敏感)。程序没有写错, 两个问题的计算也都是直接的代入求值、无法说明算法不稳定; 是问题 B 本身在 $x \approx 1$ 附近条件数很大。

(4) 良态与病态的区别:

良态问题中输入数据的微小扰动只引起输出同量级 (或更小) 的变化, 如问题 A 中 $r_y \approx r_x$;

病态问题中输入的微小扰动会被问题本身急剧放大, 输出发生巨大变化, 如问题 B 中 r_y 是 r_x 的约 10^5 倍。

这是问题的内在性质, 与用什么算法无关。问题 B 中 $y = 1/(x-1)$, 当 x 接近 1 时分母 $x-1$ 接近于 0, 此时条件数 $|x \cdot y' / y| = |x / (x-1)| \approx 1/\epsilon \approx 2.5 \times 10^5$ 极大: 分母的绝对变化 ϵ 虽小, 但相对于分母本身 (也是 ϵ 量级) 就是 $O(1)$ 的巨大相对变化, 取倒数后输出自然剧烈波动。所以 x 接近 1 时该问题对输入误差特别敏感——这是病态, 换任何稳定算法都无法消除, 只能提高数据精度或改用更高精度表示。